

Rank 7 Vulnerability Fix — AI Session Log

AES-128 Peripheral: No Fault Detection or Error Signaling

Waylon Woo with Claude Opus 4.6 (1M context)

2026-04-12

Session Log: Rank 7 Vulnerability Fix

Environment: Claude Code (Opus 4.6, 1M context, high-effort mode) **Working directory:** `~/repo/projects/AI-HDL/main` **Outcome:** Expose the internal `fault` signal in the memory-mapped status register so software can distinguish “still running” from “halted by integrity failure.”

Turn 1 — Read the security report, focus on Rank 7

User: Read rank 7 in security eval here

(Claude located `DP3_submission/security_report.pdf`, extracted all 13 pages, and surfaced the Rank 7 section.)

Claude:

Rank 7 — MEDIUM: No Fault Detection or Error Signaling

Problem (as written in the report). The chip has no way to signal that something went wrong. The `done` signal fires whether encryption completed correctly or was corrupted by a fault. `data_ready` is hardwired to 1 and never indicates an error.

Why it matters. A DFA attacker needs *both* a correct and a faulty ciphertext for the same plaintext to recover the key. If the chip always outputs data (correct or not) and always asserts `done`, every glitch attempt yields usable data. A fault-aware chip can refuse to output the faulty result (or output zeros) and deny the attacker their fault pair.

Proposed fix (report). Add a `fault_detected` output. Assert when:

- the round counter goes out of expected range;
- the FSM enters an illegal state;
- a redundant computation disagrees.

When `fault_detected` is high, output all zeros instead of ciphertext and require a full reset before the engine works again.

Turn 2 — Challenge: don’t we already have a fault signal?

User: Don’t we already have a fault signal?

(Good catch. Claude grepped for `fault` across the AES RTL and walked the `AES_control_unit + peripheral.v` wiring.)

Claude: Rank 2's fix already landed much of what Rank 7 asks for. Audit of the current state:

Already in place (from Rank 2, commit 491d2dc):

- AES128/AES_control_unit.v:9,37 — fault output, asserted while the FSM sits in a sticky FAULT state.
- AES128/AES_control_unit.v:31-35 — integrity detectors for both the one-hot state encoding (state_ok) and the count + count_shadow == 22 invariant (count_ok). Any failure latches FAULT.
- AES128/AES_control_unit.v:71,78 — done is force-cleared in FAULT, so no spurious done on faulty ciphertexts.
- peripheral.v:47-50 — engine_reset = core_reset | fault wipes state_reg, sub_out_hi, and round_key the moment a fault is detected.
- FAULT is sticky across soft reset; only a hard reset (reset_n) clears it.

The gap Rank 7 still has. fault is *not software-visible*. The status register in AES_memory.v:158 only exposes status_done:

```
6'h01: data_out = {31'b0, status_done}; // status reg: bit0 = done
```

So software observing a hung encryption sees done == 0 and cannot distinguish two very different conditions:

- “encryption is still in progress — keep polling,” vs.
- “FSM/counter integrity failure — the engine has halted and will never finish until a hard reset.”

The minimal fix. Add fault to the status register as bit 1. No new detection logic is needed — Rank 2 already built all of it — only the plumbing from AES_control_unit.fault out through AES_memory.

Turn 3 — Apply the fix

User: Great do that

(Claude made three surgical edits.)

1. AES128/AES_memory.v — add fault input, expose in status reg

```
module AES_memory (
    ...
    input [127:0] ciphertext,
    input done,
    input fault,                // NEW
    output reg [31:0] data_out,
    ...
);

...

// status reg: bit0 = done, bit1 = fault (Rank 7).
// Software polls bit0 for completion and bit1 to distinguish
// "still running" from "halted by FSM/counter integrity
// failure" (see AES_control_unit FAULT state).
6'h01: data_out = {30'b0, fault, status_done};
```

2. peripheral.v — wire fault into the memory instance

The fault wire already exists in peripheral.v:47 (it feeds engine_reset). Just connect it to the new memory port:

```

AES_memory aes_memory_inst (
    ...
    .start(start),
    .done(done),
    .fault(fault),                // NEW
    .encryption_engine_data(encryption_engine_data),
    .original_key(original_key),
    .reset_s(reset_s)
);

```

3. AES128/testbenches/AES_memory_tb.v — drive fault + assert it

Add a `fault` stimulus reg (default 0 so existing tests are unaffected), connect it to the DUT, and append a new TEST 6 that toggles the input and reads 0x01.

```

reg fault = 1'b0;
...
AES_memory dut (
    ...
    .done(done),
    .fault(fault),
    .data_out(data_out),
    ...
);
...
// === TEST 6: fault exposed in status reg bit 1 ===
fault = 1'b0; done = 1'b0;
check32(6'h01, 32'h00000000);

fault = 1'b1;
check32(6'h01, 32'h00000002);

fault = 1'b0;
check32(6'h01, 32'h00000000);

```

Turn 4 — Verify

Compile and run both the unit-level memory tb and the full-chip integration tb.

Memory tb (unit test)

```

iverilog -g2012 -o outputs/mem_test \
    AES128/testbenches/AES_memory_tb.v AES128/AES_memory.v
vvp outputs/mem_test

=== TEST 6: fault exposed in status reg bit 1 ===
PASS addr 0x01 => 0x00000000
PASS addr 0x01 => 0x00000002
PASS addr 0x01 => 0x00000000

=====
PASSED: 63
FAILED: 0
ALL TESTS PASSED

```

=====

63 assertions pass (60 pre-existing + 3 new Rank 7 checks). Tests 1–5 (basic read/write, auto-start, double-buffering, key reuse, Rank 5 interlock) still pass — no regression.

Peripheral tb (integration test)

```
iverilog -g2012 -o outputs/periph_test \  
    AES128/testbenches/AES_peripheral_tb.v peripheral.v \  
    AES128/AES_memory.v AES128/AES_control_unit.v \  
    AES128/AES_encryption_engine.v AES128/AES_key_engine.v \  
    AES128/sbox_lookup.v  
vvp outputs/periph_test
```

=====

```
PASSED: 9  
FAILED: 0  
ALL TESTS PASSED
```

=====

Full-chip elaboration succeeds with the new port wiring, and all nine integration-level assertions still hold.

Before / After

Property	Before	After
FSM / counter integrity detection	Present (Rank 2)	Present (Rank 2)
Engine halt + state wipe on detected fault	Present (Rank 2)	Present (Rank 2)
done suppressed in FAULT	Yes	Yes
Software visibility of fault state	None	Status reg bit 1 (0x01)
Software can distinguish “busy” from “halted”	No	Yes

Summary of artifacts

Files changed

File	Change
AES128/AES_memory.v	Added input fault port; status reg 0x01 returns {30'b0, fault, done}
peripheral.v	Wired fault from aes_control_unit_inst into aes_memory_inst
AES128/testbenches/AES_memory_tb.v	Added fault stimulus reg; TEST 6 (3 new asserts) verifies status bit 1

Verification

- Memory tb: **63/63 asserts pass** (60 existing + 3 new Rank 7 checks).
- Peripheral tb: **9/9 asserts pass** (full-chip integration, no regression).

Process note

Rank 7 in the security report was written as if no fault-detection machinery existed at all. By the time we got to it, Rank 2's fix had already landed the hard part — integrity detectors, sticky FAULT state, `done` suppression, engine-state wipe. The *detection* side of Rank 7 was already closed; only the *observability* side remained.

The useful lesson: when fixes land sequentially against a ranked vulnerability list, earlier fixes can partially absorb later items. Re-reading a later item in light of what is already in the tree is cheaper than re-implementing machinery that already exists, and a ten-second grep for `fault` caught this before a single line of redundant RTL was written.

Credit to the user for pushing back with “don’t we already have a fault signal?” — without that nudge, the default would have been to build the whole detection stack a second time.

Out of scope

- **Clearing stale ciphertext on fault.** `data_out_mem` currently retains whatever was latched from the previous successful encryption. A fault halts the engine but does not wipe the prior block's output. The report's “output all zeros instead of ciphertext” recommendation could be met by also zeroing `data_out_mem` on `fault`, but this is a minor hardening: the stored value is not the faulty computation result, and software can already detect the fault via bit 1 of the status register before trusting any `data_out_mem` read.
- **Dedicated `fault_detected` output pin.** The report proposed a top-level output. The status-register bit is sufficient for software to query and avoids adding to the TinyQV peripheral pin budget. A dedicated pin would only matter for board-level monitoring, which is not in scope for Tiny Tapeout.
- **Ranks 3, 4, 6, 8** from the security report remain unaddressed in this session. See `DP3_submission/security_report.pdf` for details.